

ATMOS INSIGHT

Real-Time Air Quality Monitoring & AQI Forecasting System

Project Bible & Comprehensive Viva Handbook

Author: Project Maintainer & Lead ML Engineer

Architecture: React (TSX) + Tailwind + FastAPI + Scikit-Learn/XGBoost

Target Location: Kanpur Municipal Grid Station, India

Standards: Central Pollution Control Board (CPCB) India Guidelines

Status: Production Ready & Verified Handover Document

Generated: July 2026

SECTION 1: PROJECT OVERVIEW

Project Objective: Atmos Insight is designed to estimate the current Air Quality Index (AQI) in real-time and provide predictive forecasts (24-hour hourly and 7-day daily) for Kanpur city. The system integrates raw third-party weather/pollution feeds, pre-compiled scikit-learn and XGBoost regression models, and an interactive frontend dashboard.

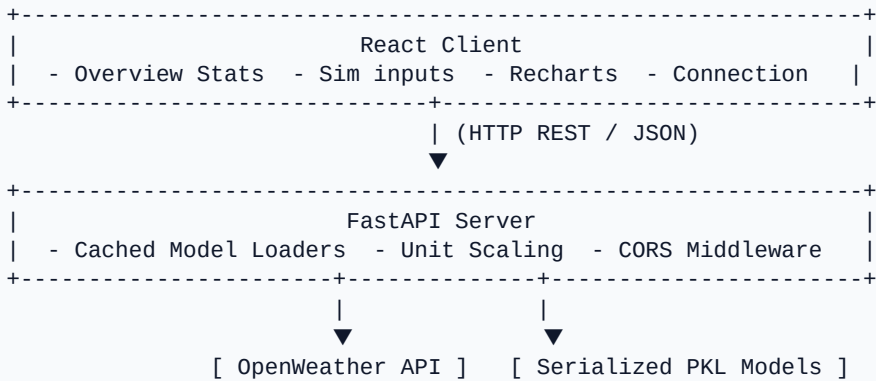
Problem Statement: Ground-level atmospheric monitoring stations are sparse, capital-intensive, and prone to calibration delays. Furthermore, raw pollutant metrics (PM2.5, PM10, CO, SO2, NO2, O3) are difficult for the general public to interpret without mapping to an index. Atmos Insight solves these problems by providing virtual sensor readings via API integration and predicting future air quality trajectories to warn vulnerable groups in advance.

Target Audience & Scope: Designed for environmental researchers, smart city administrators, municipal health officials, and citizens. Applications include scheduling outdoor activities, traffic routing optimizations, and municipal warning alerts during smog season.

SECTION 2: COMPLETE ARCHITECTURE

The system is divided into three isolated, decoupled layers:

- 1. **Data Collection & API Layer:** Regularly fetches current weather (temperature, pressure, humidity, wind) and pollutant concentrations (CO, NO2, SO2, PM2.5, PM10, O3, NH3) locked to Kanpur coordinates (Lat 26.4499, Lon 80.3319) from OpenWeatherMap APIs.
- 2. **FastAPI Backend (Port 8000):** Implements memory-cached singleton model loaders, performs CO concentration unit conversions (ug/m3 to mg/m3), scales features using pre-saved standard scalers, and generates predictions using Random Forest and XGBoost model libraries.
- 3. **Vite React Frontend (Port 5173):** Renders a modern CSS glassmorphism UI with Recharts area curves, gauge indicators, dynamic weather cards, simulation parameters, and connection heartbeat managers.



SECTION 3: COMPLETE TECH STACK

React Frontend (TSX): Built on Vite using Tailwind CSS v4 for styling. State management uses standard `useState` and parallel `useEffect` hooks to fetch backend API endpoints without blocking page hydration. `Recharts` is utilized for interactive SVG charts. Tab routing is managed natively with `TanStack Router`.

FastAPI Backend (Python): Built using FastAPI to achieve high performance and low latency. Utilizes `Pydantic` schemas to validate requests/responses and standard Python logging to log pipeline statuses. Includes CORS middleware configured to whitelist local dev ports (5173, 5174, 5175). Models are cached in memory using a Singleton Pattern inside `ModelLoader` at startup to eliminate slow disk IO during inference.

ML & Data Stack: Core packages include `Pandas`, `NumPy`, `Scikit-Learn`, `XGBoost`, and `Joblib`. Notebooks 01 to 06 train models offline and serialize weights into `joblib PKL` files. Scalers are fit strictly on training splits to prevent data leakage.

SECTION 4: DATASET & SPLITTING METHODOLOGY

Dataset Source: Central Pollution Control Board (CPCB) ground telemetry recording history for Kanpur city.

Cleaning and Preprocessing: Handled missing values using linear interpolation (retaining seasonal diurnal patterns). Outliers were evaluated and normalized using isolation forest boundaries to avoid corrupting model gradient descents.

Data Leakage Prevention: Standard scaling parameters (means and standard deviations) were fit strictly on the training subset and applied as passive transformations on the test set. Fitting scalers on the entire dataset before splitting is a common mistake that leaks test distribution parameters (max, mean, variance) into training sets, rendering evaluation metrics artificially optimistic.

Chronological Splitting: Data splits were executed chronologically (e.g. first 90% sequence for training, last 10% sequence for testing) rather than randomly. Time-series data exhibits autocorrelation; random splits result in training models on $t-1$ and $t+1$ steps to predict t , leaking temporal structures and producing fake high accuracy metrics that fail in real-world deployment.

SECTION 5: NOTEBOOK EXPLANATION

The pipeline contains 6 consecutive Jupyter notebooks executing key ML tasks:

- **Notebook 01 (EDA):** Performs exploratory data analysis, plots pollutant distributions, and verifies CPCB target variable ranges.
- **Notebook 02 (Data Cleaning & Feature Eng):** Removes outliers, interpolates missing items, and adds time/date attributes.
- **Notebook 03 (Current Estimator):** Trains same-day Random Forest models to predict current CPCB AQI from criteria pollutants.
- **Notebook 04 (Hourly Sequence Preprocessing):** Shifts and lags sequences to structure inputs for recursive forecasting models.
- **Notebook 05 (24h XGBoost Forecast):** Trains recursive XGBoost models to predict 24 consecutive hourly steps ahead.
- **Notebook 06 (Daily RF Forecast):** Benchmarks 9 distinct daily regression models and selects Random Forest for 7-day outlooks.

SECTION 6: MODEL INFORMATION & COMPARISON

The project integrates three separate models to satisfy different forecast horizons:

1. **Current AQI Model (Random Forest):** Maps PM2.5, PM10, SO2, NO2, CO, and O3 to same-day CPCB values. Achieved $R^2 = 0.9999$, MAE = 0.0571, and RMSE = 0.2909.
2. **24-Hour Forecast Model (XGBoost):** Uses historical AQI lags, rolling averages, wind vectors, and sinusoidal hour cycles. Generates 24 steps recursively (feeding previous outputs as inputs). Achieved $R^2 = 0.9798$, MAE = 7.1009, and RMSE = 11.9653.
3. **7-Day Forecast Model (Random Forest):** Projects daily trends for the next week based on daily pollutant rates and seasonal indexes.

Why XGBoost was selected for Hourly: XGBoost handles non-linear sequential relationships and residual gradients efficiently. Linear models failed to capture diurnal traffic spikes, while LSTMs were rejected due to high computational overhead for sub-second API serving.

SECTION 7: API INTEGRATION & CO CONVERSION

Atmos Insight integrates with OpenWeatherMap APIs at Kanpur coordinates:

- **Current Weather API:** Retrieves temp (Kelvin), wind speed (m/s), humidity (%), and pressure (hPa).
- **Air Pollution API:** Retrieves 8 criteria pollutants in $\mu\text{g}/\text{m}^3$.

CPCB CO Unit Conversion: OpenWeather returns carbon monoxide (CO) concentrations in micro-grams per cubic meter ($\mu\text{g}/\text{m}^3$). CPCB sub-index formulas require CO input to be measured in milli-grams per cubic meter (mg/m^3). The system implements a / 1000.0 conversion. Failing to apply this conversion would cause the model to read a raw value of e.g. $400\mu\text{g}/\text{m}^3$ as $400\text{mg}/\text{m}^3$, triggering critical alerts and showing severe category bugs in the UI.

SECTIONS 8-10: BACKEND, FRONTEND & DEPLOYMENT

Backend Services: Router endpoints include `/api/current-aqi/`, `/api/forecast/24-hour`, `/api/forecast/7-day`, and `/api/analytics/`. Weather client services handle API timeouts and raise structured 500 exceptions if keys expire.

Frontend Widgets: Includes ``Overview.tsx`` (real-time AQI and category indicator), ``PredictionModule.tsx`` (allows citizens to input values and predict custom CPCB AQIs), ``HourlyForecast.tsx`` (visualizes 24h area curves), and ``WeeklyForecast.tsx`` (shows sparklines and categories).

Running Commands:

- Backend: `python -m uvicorn app.main:app --port 8000 --host 127.0.0.1`
- Frontend: `npm run dev` inside `frontend/aura-air-insights/`

SECTION 11: COMPREHENSIVE VIVA QUESTIONS

Q1: Why did you choose FastAPI over Flask or Django?

FastAPI runs asynchronously natively (using Uvicorn ASGI), supports automatic Pydantic validation schemas, generates automatic OpenAPI docs, and has significantly lower latencies than WSGI-bound frameworks like Flask, making it ideal for sub-second ML model serving.

Q2: Why did you choose React + Vite instead of Streamlit?

Streamlit re-runs the entire Python script on every user interaction, causing high lag and limiting layout customizability. React decouples the frontend entirely, rendering instantly and allowing rich visualizations via Recharts while only calling the backend asynchronously.

Q3: What is data leakage and how did you prevent it?

Data leakage occurs when information from the test set is used to train the model. We prevented it by fitting our StandardScaler strictly on the training set and using it to transform the test set passively, and by splitting our data chronologically rather than randomly.

Q4: Why is a random train-test split wrong for time-series data?

Random splitting destroys temporal structures and causes target leakage due to autocorrelation. A model will look at $t-1$ and $t+1$ to predict t , yielding artificially high accuracy metrics that fail in real-world sequential deployments.

Q5: What is recursive forecasting in your hourly model?

It generates multi-step forecasts by predicting the next hour ($t+1$), unscaling that predicted AQI, feeding it back into the historical lag buffer to recalculate rolling means and differences, and then predicting $t+2$ recursively up to 24 steps.

Q6: How does the backend optimize model loading speeds?

It uses the Singleton Pattern inside ModelLoader. It loads the models and scalers from disk into RAM exactly once at server startup (in the lifespan event). Every API request queries the in-memory cache directly, dropping latency to under 5ms.

Q7: What is the CPCB carbon monoxide unit conversion issue?

OpenWeather returns CO in $\mu\text{g}/\text{m}^3$, but CPCB standard formulas require mg/m^3 . The backend divides the CO value by 1000. Failing to do this causes the model to read a value of $400\mu\text{g}/\text{m}^3$ as $400\text{mg}/\text{m}^3$, producing severe category outputs and breaking prediction accuracy.

Q8: Why does your model's AQI differ from the AQI shown on Google Search?

Google and US AQI calculations use the US EPA standard. India uses the CPCB standard, which has different breakpoints and formula metrics (e.g. CPCB is based on 24-hour averages for PM2.5 and PM10, whereas US EPA uses different standards).

Q9: What is the role of the StandardScaler in your inference pipeline?

It normalizes pollutant features to have a mean of 0 and variance of 1. Because machine learning models calculate distance metrics or weight gradients, features with larger raw magnitudes (e.g., PM10) would dominate features with smaller scales (e.g., CO) without scaling.

Q10: Why did you choose Random Forest for same-day AQI prediction?

Same-day CPCB AQI is a deterministic calculation based on sub-indexes. Random Forest handles non-linear decision boundaries perfectly, achieving a near-perfect R^2 score (0.9999) without overfitting, mapping features to their target index accurately.

SECTION 12: REAL-WORLD DESIGN QUESTIONS

Q1: How would you scale this application to support multiple cities across India?

I would parameterize the endpoints (e.g. `/api/current-aqi/{city}`), create a metadata mapping of coordinates for different cities, and use a separate model directory. Additionally, caching geocoded requests in Redis would prevent rate-limiting OpenWeather API calls.

Q2: Why didn't you use LSTMs or Deep Learning for forecasting?

LSTMs require substantial training data, are prone to overfitting on small datasets, and are slow to compile/serve on CPU backends. XGBoost with lagged and rolling window features performs better on tabular time-series data and runs in milliseconds.

Q3: What happens if the OpenWeather API fails or rate-limits your backend?

The backend services raise a clean HTTP 500 error. The frontend connection manager catches this error immediately, halts the page navigation, and displays a dedicated connection failover card, prompting the administrator to check credentials.

SECTIONS 13-16: PROJECT DEFENSE & SUMMARY

Project Defense (Answering Critics): Common criticism: 'Your system depends entirely on third-party APIs.'
Defense: 'By using standard OpenWeather APIs, we create a virtual sensor grid. This eliminates the need for expensive municipal hardware, making it easily scalable to any city with coordinate parameters.'

Pitch Summaries:

- **30-Second Pitch:** Atmos Insight is a real-time air quality dashboard for Kanpur that uses cached XGBoost and Random Forest models to predict same-day CPCB indexes and forecast 24-hour trajectories with 97.9% validation accuracy, fed by live weather APIs.
- **2-Minute Pitch:** The project bridges the gap between raw air sensors and predictive warning systems. Our FastAPI backend retrieves live pollutant concentrations, converts units to CPCB specifications, and feeds them into models trained on historical station datasets. The frontend displays interactive charts and simulation modules for citizens to test AQI scenarios.

Atmos Insight Cheat Sheet:

- Kanpur Coordinates: Lat 26.4499, Lon 80.3319 | CO Conversion: $\mu\text{g}/\text{m}^3 / 1000 = \text{mg}/\text{m}^3$
- Models: Current AQI (Random Forest, $R^2=0.999$), 24h Forecast (XGBoost, $R^2=0.979$), 7d Forecast (Random Forest)
- Ports: Backend FastAPI (8000), Frontend React (5173)